

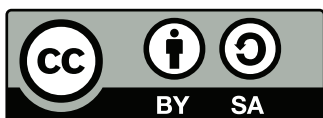


Universidad Complutense de Madrid

Documentación del software desarrollado para la práctica de control de un motor de cc

Juan Jiménez

29 de julio de 2026



El contenido de estos apuntes está bajo licencia Creative Commons Attribution-ShareAlike 4.0
<http://creativecommons.org/licenses/by-sa/4.0/>
©Juan Jiménez

Índice general

1. Primeros pasos	2
1.1. Software necesario	2
2. El entorno de desarrollo de Visual Studio Code (VSC)	4
2.1. Instalación de las extensiones STM32Cube	5
2.2. Instalación de las extensiones para Python	5
3. Configuración de la placa con STM32CubeMX	6
3.1. Placa en modo debug y reseteo del pin PB3	6
3.2. Configuración del Reloj.	6
3.3. Configuración de un puerto serie (UART), a través del puerto USB de la placa.	7
3.4. Configuración del resto de pines para el control del motor	9
3.5. Habilitar un timer para que lea periódicamente el número de pasos (Encoders) que ha avanzado el motor.	10
3.6. Instalar y activar CMSIS-DSP	10
3.7. Definir CMake para la compilación y el enlazado y obtener el esqueleto del código del proyecto	11
4. Programación del Microcontrolador en C	12
4.1. Estructura del proyecto	12
4.2. main.h y main.c	13
4.2.1. main.h	13
4.2.2. main.c	14
4.3. motor.h y motor.c	20
4.3.1. motor.h	20
4.3.2. motor.c	21
5. Programación en Python de un monitorizador.	25
5.1. MotorDashboard	25

Índice de figuras

2.1. Ventana de Visual Studio Code	5
3.1. Ventanas de inicio y selección de placa	7
3.2. Vista de la pestaña de configuración del reloj, con el reloj configurado a 100MHz.	8
4.1. Arquitectura modular y síncrona del software de control del motor sobre la placa STM32.	16
5.1. Vista del panel de control desarrollado para interactuar con el motor	26

Índice de Tablas

3.1. Mapa de conexiones nativas entre el sistema electromecánico y la placa STM32 Nucleo.	9
---	---

Resumen

Estas notas son complementarias al guión de la práctica y explican todo el proceso de construcción del software empleado en la práctica. Dan información de los IDEs empleados para el desarrollo del software, de su instalación, de la configuración de la placa de control empleada y de la estructura de los programas desarrollados

Capítulo 1

Primeros pasos

1.1. Software necesario

En primer lugar, como se ha explicado ya en el guión de la práctica, vamos a emplear la placa STM32F411E, de la compañía ST, para controlar el motor. En la jerga de los programadores de microcontroladores, esta placa es conocida como la *Black Pill*. Una de sus ventajas, es que tiene un excelente soporte de software y herramientas de desarrollo.

En esta sección vamos a describir como a asumir que estamos trabajando con un ordenador personal con sistema operativo Linux, en concreto vamos a emplear Ubuntu 24.04.4 LTS.

<https://ubuntu.com/download/desktop>

Algunas herramientas básicas necesarias son:

1. Visual studio Code (VSC). Será el entorno de desarrollo que emplearemos para escribir, compilar y descargar el código que utilizará la placa ST. También servirá para desarrollar el código en Python que nos permitirá comunicarnos y controlar la placa desde el ordenador. Para instalarlo en Ubuntu, lo ideal es descargar directamente el paquete .deb, de la página web de VSC,

https://code.visualstudio.com/thank-you?dv=linux64_deb

para instalarlo ir a la línea de comandos:

```
sudo apt install ./Descargas/code_*.deb
```

Ver descripción detallada en el capítulo 2

2. STM32CubeMX: Es la herramienta gráfica oficial de ST. Sirve para configurar los pines, el reloj (clock) y generar el código base en C.

<https://www.st.com/en/development-tools/stm32cubemx.html>

3. STM32CubeCLT: Tiene todas las herramientas de ST, necesarias para compilar, enlazar etc. Para instalarlo Descargar de la página web de ST,

<https://www.st.com/en/development-tools/stm32cubecolt.html>

el que pone Debian Linux installer. Descomprimir,

```
unzip st-stm32cubecolt_1.21.0_27995_20260219_1804_amd64.deb_bundle.sh.zip
```

Dar Permisos

```
chmod +x st-stm32cubecolt_1.21.0_27995_20260219_1804_amd64.deb_bundle.sh
```

Lanzar el instalador

```
sudo ./st-stm32cubeclt_1.21.0_27995_20260219_1804_amd64.deb_bundle.sh
```

Presentará en el terminal la licencia de uso, ir hasta el final y aceptarla. Debería instalarlo en:

```
/opt/st/stm32cubeclt_1.21.0/
```

Es toolset STM32CubeCLT incluye:

- GNU C/C++ for Arm® toolchain tales como arm-none-abi-gcc(compilador), arm-none-abi-nm (symbol viewer) y otros.
 - GDB debugger cliente y servidor (Depurador). Depurar el programa ejecutándolo en la propia placa
 - STM32CubeProgrammer (STM32CubeProg) utility (Programador) Descargar el ejecutable desde el ordenador a la placa.
 - System view descriptor files (.SVD) for the entire STM32 MCU portfolio.
 - Map file associating STM32 MCUs and MCU development boards to the appropriate SVD
4. Cmake y Ninja: fundamentales para gestionar la construcción del proyecto: Librerías, dependencias, etc.
 5. ST-Link Drivers: permite la comunicación entre el ordenador y la placa de desarrollo a través de un puertos USB. Estas dos últimas herramientas se instalan directamente desde VSC. (Ver capítulo 2)
 6. Python y los modulos de Python: serial, struct, csv, sys, time, collections, datetime, pyqtgraph.
 7. (Opcional) Si se va a hacer el análisis de los datos y los modelos del motor en python, entonces es conveniente instalar también los módulos: pandas, numpy, matplotlib.pyplot y control

Para instalar STM32CubeMx, lo ideal es obtenerlo de la página web de ST,

<https://www.st.com/en/development-tools/stm32cubemx.html>

Igualmente, para STM32CubeProgrammer,

<https://www.st.com/en/development-tools/stm32cubeprog.html>

El resto de las herramientas, se pueden instalar directamente a través de Visualstudio Code, Mediante las extensiones para SMT32.

En el caso de Python, es bastante cómodo instalar directamente anaconda.

<https://www.anaconda.com/downloadP>

Varios de los módulos de Python necesarios vienen instalados por defecto con Anaconda y el resto son fácilmente instalables empleando **conda**.

Capítulo 2

El entorno de desarrollo de Visual Studio Code (VSC)

Entre las diferentes opciones existentes de IDEs (Integrated development Environments), se ha optado por emplear el programa Visual Studio Code para el desarrollo de este proyecto.

Si abrimos el programa, una vez instalado, accedemos a una ventana como la se muestra en la figura 2.1. Vamos a describir brevemente los elementos más importantes.

- Barra de actividad: Es la barra vertical localizada a la izquierda del todo de la ventana. Esta barra permite cambiar la vista del panel situado inmediatamente a su derecha. Las distintas opciones hacen referencias a tareas distintas realizables desde VSC. Así el icono superior (Explorer nos permite navegar por los directorios que tenemos abierto, seleccionar archivos para abrirlos en el editor etc. Debajo tenemos una opción de búsqueda (Search). Un control de cambios basado en git (Source control). Un botón para ejecución y depuración de código (RUN and debug), Un botón muy útil que nos permite explorar las extensiones (Extensions) disponibles, es decir configuraciones específicas para manejar distintos tipos de lenguajes de programación, etc.
- Barra Lateral, inmediatamente a la derecha de la anterior, ofrece distintas vistas y paneles, dependiendo de la actividad seleccionada en la barra de actividad. En la figura 2.1, se ve el panel correspondiente a las extensiones, porque la actividad seleccionada, ha sido precisamente el botón extensions. Se pueden recorrer las extensiones disponibles, comprobar si están o no instaladas, y seleccionarla para su instalación/desinstalación, configuración, etc.
- El editor. Ocupa el area central. Su objetivo es escribir y editar código. Es posible abrir varios editores uno al lado del otro y de este modo trabajar en varios archivos simultáneamente. En la figura 2.1, se muestra un fichero de texto que contiene la fuente el $\text{\LaTeX}$ con la que se han elaborado estas notas. A su derecha, se ve el texto resultante en un archivo pdf.
- Barra de estado (Status Bar) Situada en la parte inferior de la ventana, ofrece información contextual relacionada con el estado actual de VSC y la actividad que se está llevando a cabo.
- Paleta de Comandos (Command Palette) Se accede pulsando los comandos $\text{Ctrl} + \text{Shift} + \text{P}$, es muy útil porque permite el acceso rápido a muchos comandos y características de VSC, sin tener que navegar por menús de opciones.
- Paneles. Están localizados debajo del editor. Muestran distintas herramientas muy útiles. Terminales del sistema, Salidas para los distintos compiladores, Consolas de depuración, panel de problemas etc.
- Tabs. Situados en la parte superior del editor, permiten intercambiar la vista entre los archivos abiertos.

Una vez instalado VSC debemos seleccionar las extensiones adecuadas para poder compilar código C, para la placa, descargarlo y, en su caso, poder depurarlo.

Además, podemos también instalar las extensiones para escribir y ejecutar código en python. De esta manera, con un solo IDE, podemos generar todo el código necesario; tanto el código en C para la placa controladora como en código en Python para interactuar con la placa desde el ordenador.

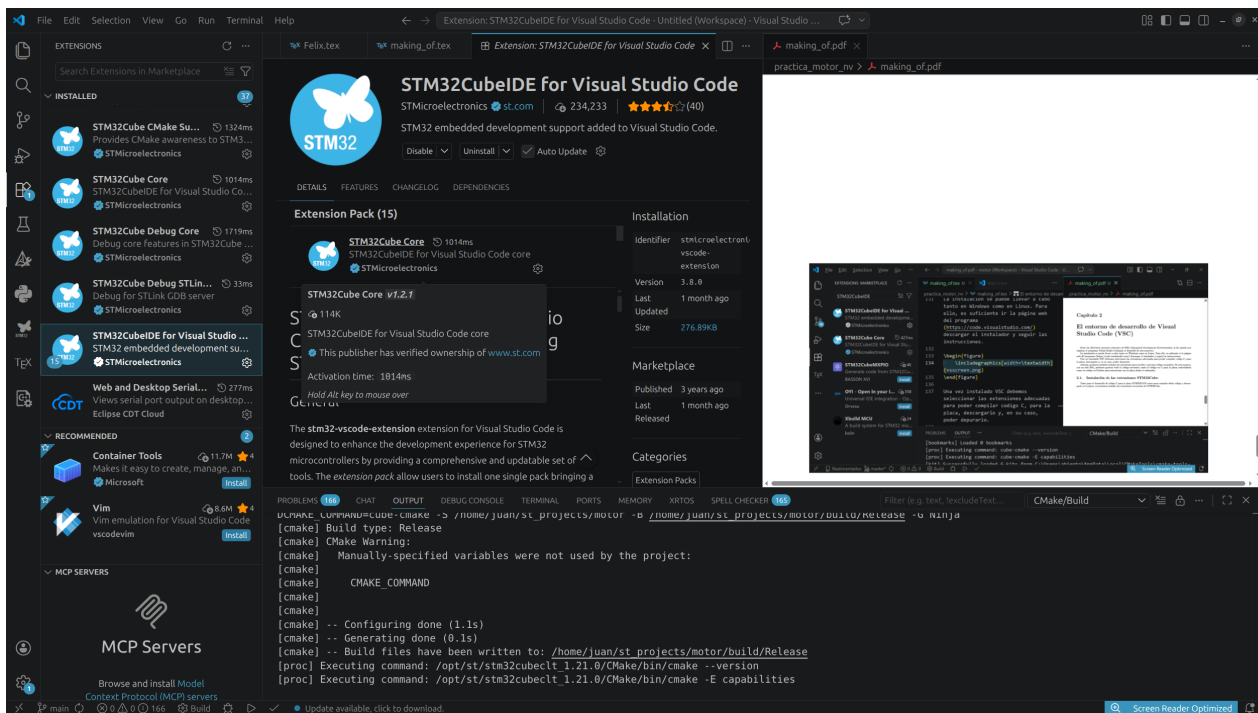


Figura 2.1: Ventana de Visual Studio Code

2.1. Instalación de las extensiones STM32Cube

Tanto para el desarrollo de código C para la placa STM32F411E como para compilar dicho código y descargarlo en la placa, necesitamos instalar las extensiones necesarias de STM32Cube.

Para hacerlo, pulsamos en la barra de actividades, el botón de las extensiones que tiene un icono con cuatro cuadros. Automáticamente aparecerá en la barra lateral las extensiones disponibles. Lo más sencillo es escribir en la barra de búsqueda la extensión que deseamos instalar: **STM32CubeIDE for Visual Studio Code**. Las extensiones de ST, empujan como símbolo una mariposa sobre un círculo azul, seleccionamos la que hemos buscado, pulsamos en el botón 'install'. Dejamos que complete la instalación. Estamos ya en condiciones de escribir, compilar y descargar código para nuestra placa ST.

2.2. Instalación de las extensiones para Python

VSC, puede emplear directamente la instalación de Python existente en nuestro ordenador. Sin embargo, como ya hemos indicado, es particularmente cómodo manejarlo desde la distribución de anaconda.

Para poder manejar python desde VSC podemos emplear la extensión 'Pylance'. El procedimiento de instalación es idéntico al que hemos visto para Las extensiones de ST. Buscamos Pylance en el buscador de las extensiones y pulsamos el botón 'Install'.

Una vez instalado, VSC reconocerá cualquier archivo con extensión .py como propio de Python y nos mostrará a, la altura de los tabs, un triángulo verde. Si lo pulsamos ejecutará, en el terminal, disponible en los paneles, el código de archivo .py abierto en el editor activo.

Es posible que en la primera ejecución nos pregunte que interprete de python queremos emplear, seleccionar el señalado como 'conda', asociado a la distribución de Anaconda.

Capítulo 3

Configuración de la placa con STM32CubeMX

STM32CubeMX, es una herramienta extremadamente útil para configurar el Hardware de nuestra placa. Conocer a fondo todas sus posibilidades está fuera de los objetivos de estas notas. Aquí nos vamos a centrar en configurar la placa para que pueda controlar nuestro motor. Lo vamos a hacer paso a paso.

Al ejecutar STM32CubeMX, se abre una primera ventana en la que se nos permite abrir proyectos ya existentes o crear proyectos nuevos figura (3.1a). Entre las opciones para crear proyectos nuevos, podemos emplear tanto el botón **ACCESS TO MCU SELECTOR** como el botón **ACCESS TO BOARD SELECTOR**. La diferencia es que en el primer caso nos selecciona el microcontrolador, y no nos configura nada por defecto. Si seleccionamos la placa el Package pgfkeys Error: I do not know the key '/tikz/fit', to which you passed '(tim4) (update) (cnt) (txdma)', and I am going to ignore it. Perhaps you misspelled it. programa sabe qué pines están conectados al led y al botón azul de usuario y además configura la emulación del puerto serie, a través del puerto USB de la placa, para comunicar la placa con el ordenador. Si empleamos la primera opción, tendremos que configurar nosotros a mano estas opciones, caso de que queramos usarlas.

Vamos a seleccionar directamente la placa núcleo que la que vamos a trabajar. Para ello pulsamos el botón **ACCESS TO BOARD SELECTOR**. Se abrirá una nueva ventana, figura (3.1b). Seleccionamos en la entrada **comercial part number**, arriba a la izquierda, nuestra placa: **NUCLEO F411RE**. Esperamos a que la seleccione y nos abrirá una pequeña ventana de diálogo en la que nos sugiere inicializar todos los periféricos con las opciones por defecto. Seleccionamos **no**. Se cierra la ventana de selección y en la ventana principal de STM32CubeMX aparecen varias pestañas y se nos muestra el pinout de la placa con los pines que ya están configurados por defecto, el color y 'fijados' con una chincheta figura (3.1c).

3.1. Placa en modo debug y reseteo del pin PB3

Lo primero que vamos a hacer, es poner la placa en modo debug. Hacemos esto para que nos deje programar la placa siempre que queramos sin necesidad de emplear el botón de reset. Para ello, desplegamos el menú **System Core**, seleccionamos la opción **SYS** y se nos abrirá un panel de configuración, figura (3.1d). En la entrada que puede **Debug**, seleccionamos **Serial Wire**. A continuación vamos a liberar el pin B3. Por defecto este pin está configurado precisamente para emplear el método de depuración **Serial Wire** y aparece marcado con las letras **SWO** (Serial Wire Output). Pero nosotros no necesitamos emplear ese modo de depuración y lo que vamos a hacer es resetearlo para poder emplearlo como pin de control del movimiento del motor (ver tabla 3.1) más adelante. Para resetearlo lo seleccionamos con el ratón el esquema mostrado en el panel *pinout view* y en el menú desplegable que aparece seleccionamos **Reset_State**. El pin se pondrá de color gris.

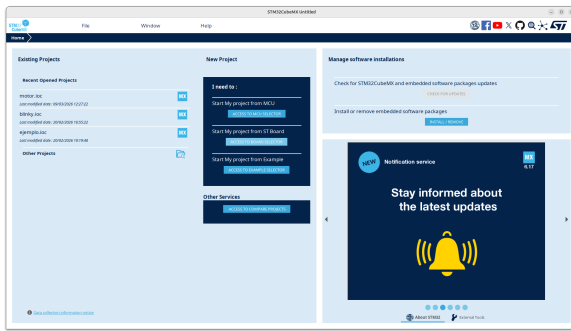
Vamos a establecer nuestras necesidades, de acuerdo con las posibilidades de la placa.

3.2. Configuración del Reloj.

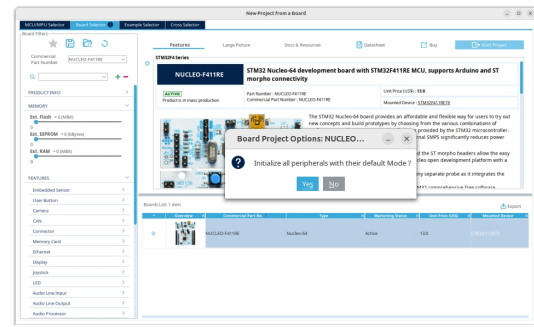
Una de las características interesantes de la *Black Pill* es su velocidad. Vamos a configurar el reloj de la placa para que trabaje a 10MHz, que es la máxima velocidad disponible.

Por defecto, la placa activa un reloj interno. Sin embargo, es mejor emplear el oscilador externo para generar la señal de reloj, ya que nos da más precisión y nos va a permitir manejar con más velocidad y precisión las comunicaciones a través del puerto serie.

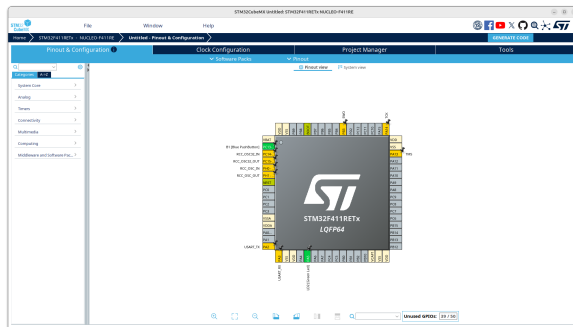
Para ello, volvemos a la categoría **System Core** y seleccionamos la entrada **RCC** (Reset and Clock Control). Buscamos la opción **High Speed Clock (HSE)**. Cambiamos "Disable" por **BYPASS Clock Source**. Hacemos esta



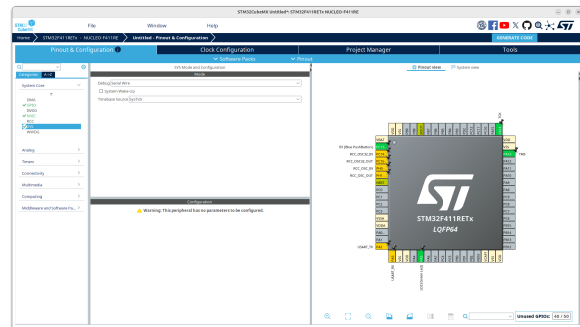
(a) Ventana inicial de STM32CubeMX



(b) Selección de la placa



(c) Layout de la placa NUCLEO F411RE



(d) Debug a Serial wire y Reseteo del pin PB3

Figura 3.1: Ventanas de inicio y selección de placa

elección porque en la placa, la señal de reloj la toma el microcontrolador directamente del programador (ST-Link). Los pines PH0 y PH1 aparecen ahora de color verde.

Configurar la frecuencia en el clock Tree Buscamos ahora, en las pestañas de la parte superior, la que aparece marcada como *Clock Configuration*. Se nos muestra un esquemático del árbol de configuración del reloj del microprocesador.

- Nos aseguramos de que la casilla **Input Frequency** aparece el número 8 (8MHz, corresponde a la frecuencia de la señal de reloj envía al microcontrolador desde fuera).
- En el multiplexor **PLL Source Mux**, marcamos la opción **HSE**.
- En la casilla **HCLK (MHz)**, escribimos 100 (que es la velocidad máxima de la F411).
- Pulsamos **Enter** y dejamos que CubeMX ajuste los divisores M,N,P,Q.

La figura 3.2 muestra una imagen de la pestaña *Clock Configuration*. Con el reloj ajustado a 100mHz.

3.3. Configuración de un puerto serie (UART), a través del puerto USB de la placa.

Para poder interactuar con el motor desde un ordenador, la que vamos a hacer es comunicarnos con la placa a través de un puerto serie.

El conector USB de la placa Núcleo F411RE que usaremos para programarla y darle energía ya tiene un conversor de puerto serie integrado (a través del chip ST-LINK que está en la parte superior de la placa). Físicamente, este chip está conectado a los pines PA2 (Transmisor - TX) y PA3 (Receptor - RX) de tu procesador, que corresponden al periférico USART2.

Para configurarlo, en la ventana del STM32CubeMX, volvemos a seleccionar la pestaña de la izquierda *Pinout & Configuration*. En el menú de la izquierda, desplegamos la categoría *Connectivity* y hacemos clic en USART2.

En la ventana central (Mode), cambiamos *Disable* por *Asynchronous*. Los pines PA2 y PA3 cambian ahora a verde

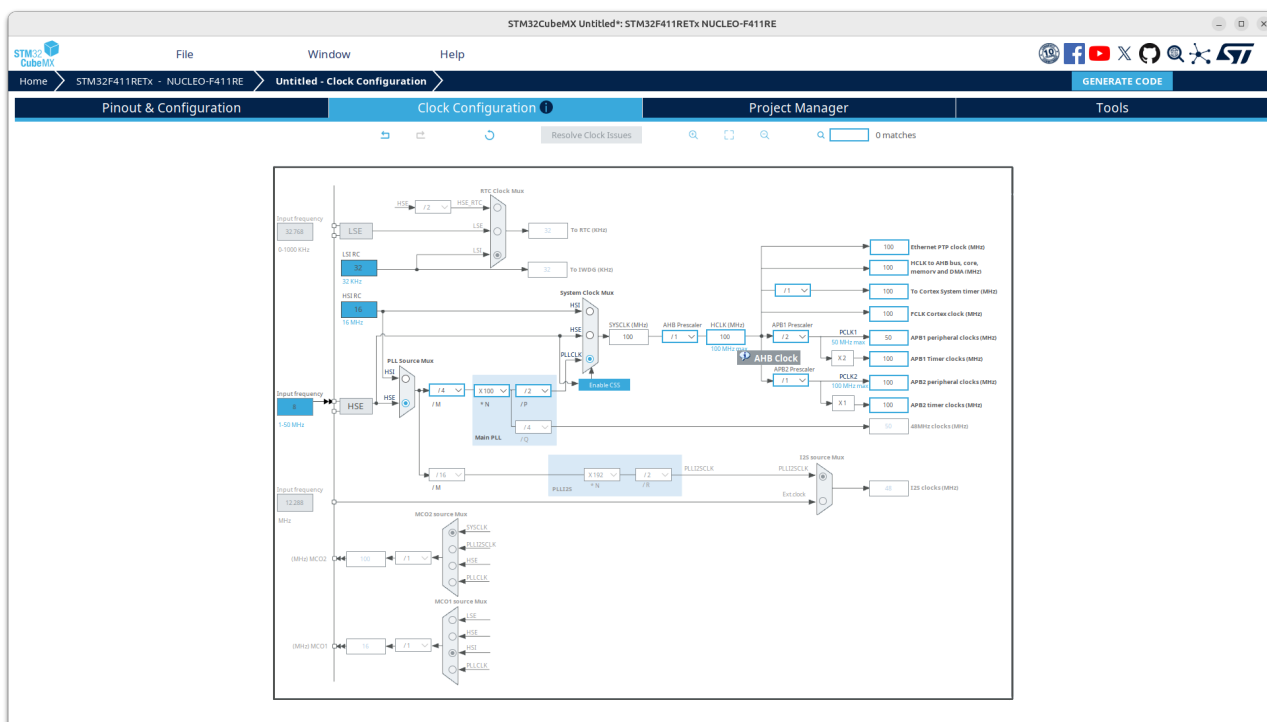


Figura 3.2: Vista de la pestaña de configuración del reloj, con el reloj configurado a 100MHz.

Abajo, en la pestaña *Parameter Settings*, la velocidad (Baud Rate) por defecto es 115200 Bits/s. Lo dejamos así, es la velocidad estándar. Simplemente habrá que tener en cuenta ese dato en la aplicación en Python que emplearemos en el ordenador para comunicarnos con la placa.

Además habilitamos una interrupción que permita a la placa recibir comandos desde el ordenador. Para ello, bajamos al panel *Configuration*, seleccionamos la pestaña **NVIC Settings** y marcamos la casilla *USART2 global interrupt*.

Por último vamos a habilitar dos canales de DMA (Acceso directo a memoria) tanto para la recepción como para el envío de datos. Esto es necesario ya que, como veremos más adelante, vamos a configurar un timer (TIM4), para que fije el ritmo al que actúa el control del motor a 1KHz. Este ritmo es imposible de seguir por el puerto serie, no podemos por tanto emplearlo para enviar datos, o bloquearemos mientras envía el control del motor. De hecho, vamos a enviarlos a un ritmo de 10MHz. Los canales de DMA desacoplan la tarea de envío/recepción de la CPU, esta sencillamente *delega* la tarea de copiar los datos desde/a memoria, al hardware del puerto serie y no se bloquea o interrumpe la recepción de datos.

Para habilitar el uso de DMA por parte del USART lo que hacemos es, en *configuración*, señalar la pestaña *DMA settings*.

A continuación, pulsamos el botón *add* seleccionamos USART2_TX. COn esto definimos el canal de DMA que va a usar el puerto YSART2 para sus envíos (TX). Completamos la configuración,

1. Direction: Memory To Peripheral (de la RAM a la UART).
2. Priority: low
3. Mode: Normal. (Existe un modo Circular, pero como tu código procesa "paquete a paquete", el modo Normal encaja perfecto con tu lógica actual).
4. Increment Address: Peripheral desmarcado (es un solo registro fijo), Memory marcado (para que vaya rellenando tu estructura byte a byte).
5. Data Width: Byte / Byte.

Para habilitar un canal de DMA para la recepción de datos, procedemos igual. Pulsamos el botón add y ahora seleccionamos USART2_RX. La configuración es exactamente la misma, salvo para la dirección (Direction) que ahora será: Peripheral To Memory (de la UART a la RAM) y la prioridad que ahora seleccionaremos: high.

3.4. Configuración del resto de pines para el control del motor

Reproducimos a continuación la tabla del capítulo 2 del guión de la práctica, en que se describen los pines que se emplearán para suministrar potencia al motor y leer las señales de los encoders.

Componente	Señal Motor/Driver	Pin STM32	Función del Periférico (STM32CubeMX)
Encoder	Canal A (Fase A)	PA0	TIM2_CH1 (Modo Encoder)
	Canal B (Fase B)	PA1	TIM2_CH2 (Modo Encoder)
Driver Potencia	PWM (Voltaje)	PB4	TIM3_CH1 (Generación PWM)
	DIR (Sentido)	PB3	GPIO_Output (Salida Digital)
	Habilitar p. H ENA	PA10	GPIO_Output (Salida Digital)
	Habilitar p. H ENB	PA7	GPIO_Output (Salida Digital)

Tabla 3.1: Mapa de conexiones nativas entre el sistema electromecánico y la placa STM32 Nucleo.

Como se deduce de la tabla, la interacción con la planta se delega en gran medida al hardware:

- **Lectura del Encoder (PA0 y PA1):** A diferencia de sistemas más simples donde se programan interrupciones por software para contar los pulsos, aquí conectamos las señales en cuadratura a los canales 1 y 2 del Timer 2. Al configurarlo en *Encoder Mode*, el hardware del STM32 detecta automáticamente el sentido de giro y actualiza el registro del contador (CNT). Nuestro software simplemente leerá este registro cuando lo necesite.

Para configurarlo adecuadamente, en el panel lateral izquierdo **Categories** de la pestaña **pinout & configuration**, Seleccionamos la entrada **timers**. Elegimos el timer TIM2. Se abrirá un nuevo panel para configurarlo. En dicho panel buscamos la opción **Combined Channels**, desplegamos el menú y seleccionamos **Encoder Mode** Automáticamente nos pondrá los pines PA0 y PA1 en color verde y les asignará, respectivamente los canales CH1 y CH2 del timer 2.

- **Generación de PWM (PB4):** Se conecta al canal 1 del Timer 3. El microcontrolador generará la señal de control de potencia (*Duty Cycle*) en este pin. basándose en el valor que calculemos en nuestro bucle de control y escribamos en su registro de comparación (CCR1). Para configurarlo,

1. En la vista del microcontrolador (el dibujo del chip), buscamos el pin PB4, hacemos clic sobre él y seleccionamos TIM3_CH1. El pin se pondrá amarillo.
2. En la lista de la izquierda, seleccionamos **Timers** ->TIM3.
3. En **Clock Source**, seleccionamos **Internal Clock**.
4. En **Channel 1**, seleccionamos **PWM Generation CH1**.
5. Abajo, en **Parameter Settings**, ponemos
Prescaler: 0
Counter Period (ARR): 1000 - 1

- **Señal de Dirección (PB3):** Se configura como una salida digital estándar (**GPIO Output**). Escribir un nivel lógico alto (1) hará girar el motor en un sentido, y un nivel bajo (0) en el sentido opuesto, invirtiendo la polaridad en el puente en H. Para configurarlo, es suficiente con hacer click sobre el pin y seleccionar **GPIO_output**.

- **Habilitar ambos puentes en H (PA10, PA7)** Habilitamos los dos puentes en H para alimentar el motor a través de ambos puentes a la vez. Estos pines deben estar siempre a nivel lógico alto (1). Los configuramos de modo análogo al anterior: hacemos click sobre ellos y seleccionamos **GPIO_output**.

- **Emplear el botón azul de usuario para invertir manualmente el sentido de giro.** Realmente, esto no es necesario salvo para controlar el manual el sentido de giro. El botón azul de la placa está conectado al pin C13. Lo que vamos a hacer es asociarlo a una interrupción para que cada vez que se pulse, la placa cambie el valor del pin inmediatamente. Para ello, Hacemos click en el pin C13 y seleccionamos **GPIO_EXTI13**. Para Package pgfkeys Error: I do not know the key '/tikz/fit', to which you passed '(tim4) (update) (cnt) (txdma)', and I am going to ignore it. Perhaps you misspelled it. activar la interrupción,

1. En la lista de la izquierda, vamos a **System Core** ->NVIC.
2. Buscamos la línea que dice **EXTI line[15:10] interrupts** y marcamos la casilla **Enabled**.

3.5. Habilitar un timer para que lea periódicamente el número de pasos (Encoders) que ha avanzado el motor.

La lectura de encoders se lleva a cabo a muy alta velocidad. Esto dará robustez a los controladores que desarrollaremos en la práctica. Vamos a habilitar un tercer timer (TIM4) para que, genere una interrupción a tiempo fijo y, de este modo decidir cuando actua la acción de control.

1. Volvemos de nuevo al panel de la izquierda y en **Timers** seleccionamos TIM4.
2. En el panel que se despliega inmediatamente a su derecha marcamos la casilla **Internal Clock**
3. Elegimos el ritmo a que llamamos a nuestra acción de control $0,001s$, es decir vamos a emplear para el control una frecuencia de $1KHz$. Para ajustar la frecuencia, empleamos la expresión,

$$F_{\text{interrupción}} = \frac{F_{\text{reloj}}}{(PSC + 1) \times (ARR + 1)}.$$

Hemos ajustado el reloj de nuestro microcontrolador a $100MHz$. Así que si ajustamos el PSC (Prescaler) a $100 - 1$, estaremos dividiendo la frecuencia del reloj entre 100. Si además ajustamos el ARR (Counter period) a $1000 - 1$, volvemos a dividir la frecuencia del reloj entre mil, de modo que la frecuencia de la interrupción quedará $1e8/1e2/1e3 = 1000Hz$.

Bajamos **configuration** y en la pestaña **Parameter Settings**. Buscamos la sección **Counter settings**. En **Prescaler** escribimos $100 - 1$ y en **Counter Period** $1000 - 1$.

4. Por último, para activar la interrupción, en la pestaña **NVIC Settings** Marcamos la casilla **TIM4 Global interrupt**.

3.6. Instalar y activar CMSIS-DSP

Por último, antes de generar nuestro proyecto, vamos a configurarlo para que use la librería CMSIS-DSP (Cortex Microcontroller Software Interface Standard - Digital Signal Processing).

Es una suite de librerías desarrollada directamente por ARM (los creadores de la arquitectura de tu STM32) y está diseñada específicamente para exprimir al máximo los procesadores Cortex-M.

Incluye un amplio número de funciones: Basic math functions, Fast math functions, Complex math functions, Filtering functions, Matrix functions, Transform functions, Motor control functions, Statistical functions, Support functions, Interpolation functions, Support Vector Machine functions (SVM), Bayes classifier functions, Distance functions, Quaternion functions.

En CMSIS-DSP, las rutinas está escrita usando instrucciones en ensamblador específicas de ARM (como multiplicaciones y acumulaciones en un solo ciclo de reloj - MAC) y usa la FPU (Unidad de Coma Flotante) por hardware.

En nuestro caso, las vamos a emplear para implementar un PID y para realizar operaciones matriciales que nos permitan emplear control por realimentación en variables de estado estimadas.

Para instalar las librerías CMSIS-DSP, seleccionamos en la barra de menú superior de la pestaña **Pinout & configuration**, la opción que **Software Packs** y hacemos clic en **Select Components**. Se abrirá una ventana nueva con una lista de librerías. (Nota: Si es la primera vez que se abre, puede que tarde unos segundos en conectarse a los servidores de ST para descargar el catálogo).

1. Desplegamos la pestaña **ARM.CMSIS**. Buscamos la fila que dice **CMSISDSP**.
2. En la columna de la derecha (llamada Selection), hacemos clic y cambiamos la opción a **Source**.
3. Hacemos clic en OK para cerrar esa ventana.
4. Volvemos a la pantalla principal del CubeMX. En la columna de la izquierda, abajo del todo seleccionamos **Software Packs**.
5. La desplegamos y hacemos clic en **ARM.CMSIS**.
6. En el panel central, aparecerá la opción **CMSIS DSP**. Activamos su casilla.

3.7. Definir CMake para la compilación y el enlazado y obtener el esqueleto del código del proyecto

Por último, dado que vamos a utilizar las herramientas de Visualstudio Code para compilar nuestro código, debemos añadir a nuestro proyecto las instrucciones para hacerlo empleando **CMAKE**, ya que esta es en Visualstudio la opción por defecto.

Para ello, seleccionamos la pestaña **Project Manager**. en la sección **Project Name** damos un nombre al proyecto, por ejemplo "motor". En la sección **Toolchain / IDE**, cambiamos la opción a **CMake**.

Con esto, hemos completado la configuración de nuestro proyecto. Solo queda pulsar el botón **GENERATE CODE**, situado arriba a la derecha de la ventana principal de CubeMX, para que se generen los archivos del lo que podríamos llamar el esqueleto de nuestro código. Creará una nueva carpeta con el nombre del proyecto (motor) y varias subcarpetas con todo el código generado. En la siguiente sección examinaremos en más detalle este código.

Toda la información de configuración del proyecto queda guardada en un archivo con el mismo nombre del proyecto y extensión **.ioc** (**motor.ioc**). Podemos modificar la configuración de nuestro proyecto, abriendo dicho fichero con el CubeMX. Recuperaremos todas las opciones que hemos ido seleccionando a lo largo de este capítulo y tendremos la opción de cambiarlas o añadir nuevas opciones. Cada vez que completemos una nueva configuración deberemos pulsar la opción **GENERATE CODE** para generar un nuevo fichero **.ioc** con los cambios.

A partir de ahora, para simplificar las explicaciones, nos referiremos siempre a nuestro proyecto y al directoria que contiene su código con el nombre de *motor*.

Capítulo 4

Programación del Microcontrolador en C

Una vez generado el proyecto con STM32CubeMX. Tenemos la estructura básica para controlar nuestro motor, pero ahora es necesario escribir el código. Para hacerlo, abrimos el directorio (open folder) motor en Visual Studio Code.

4.1. Estructura del proyecto

La carpeta motor está configurada siguiendo al estructura que se muestra en el siguiente árbol de directorios:

```
motor/
├── .settings/
├── .vscode/
├── build/
├── cmake/
├── Core/
│   ├── Inc/
│   │   ├── main.h
│   │   ├── motor.h
│   │   ├── stm32f4xx_hal_conf.h
│   │   └── stm32f4xx_it.h
│   └── Src/
│       ├── main.c
│       ├── motor.c
│       ├── stm32f4xx_hal_msp.c
│       ├── stm32f4xx_it.c
│       ├── syscalls.c
│       ├── system.c
│       └── system_stm32f4xx.c
├── Drivers/
├── .clangd
├── .gitignore
├── .mxproject
├── .project
├── CMakeLists.txt
├── CMakePresets.json
├── motor.ioc
├── Motor.py
├── registro_motor_20260506_171311.csv
├── startup_stm32f411xe.s
└── STM32F411XX_FLASH.ld
```

Todos los directorios y ficheros que comienzan por un punto, contienen archivos de configuración del proyecto, creados automáticamente por Visual Studio, o Git. También lo son los que aparecen en el directorio /motor creados por CMake, (CMakeLists.txt, CMakePresets.json) y el propio directorio cmake/. O los relacionados con la descarga de código a la placa, (startup_stm32f411xe.s, STM32F411XX_FLASH.ld) creados una vez se compila y descarga el código.

El directorio `build/` se genera al compilar el proyecto. De hecho, no aparece hasta que se compila por primera vez y podría borrarse antes de volver a compilar y CMake lo generaría de nuevo.

El directorio `Drivers/`, contiene las rutinas de CMSIS que emplearemos para desarrollar el código de control del motor y las rutinas de la *HAL* (Hardware (Hardware Abstraction Layer o Capa de Abstracción de Hardware) en microcontroladores es un conjunto de funciones de software que actúa como puente entre el código de aplicación y los registros físicos del hardware. Permite programar periféricos (GPIO, I2C, SPI) con comandos genéricos.

Hemos dejado casi para el final, El directorio principal de nuestro proyecto `Core/`. Este directorio contiene a su vez, otros dos `Inc/` y `Src/`. El primero contiene los archivos de cabecera (.h) y el segundo los archivos fuente en c (.c). Todos estos archivos han sido generados automáticamente por STM32CubeMX. Los archivos `main` y `motor`, ofrecen simples esqueletos en los deberemos añadir el código necesario para programar nuestra placa. Vamos a verlos en detalle en las proximas secciones.

4.2. main.h y main.c

4.2.1. main.h

Vamos a empezar por los archivos principales (main). El archivo de cabecera, `main.h`, es un archivo breve. Lo vamos a usar tal cual lo genera STM32CubeMX.

```
1  /* USER CODE BEGIN Header */
2  /**
3   * *****
4   * @file           : main.h
5   * @brief          : Header for main.c file.
6   *                  This file contains the common defines of the application.
7   * *****
8   * @attention
9   *
10  * Copyright (c) 2026 STMicroelectronics.
11  * All rights reserved.
12  *
13  * This software is licensed under terms that can be found in the LICENSE file
14  * in the root directory of this software component.
15  * If no LICENSE file comes with this software, it is provided AS-IS.
16  *
17  * *****
18  */
19  /* USER CODE END Header */
20
21  /* Define to prevent recursive inclusion -----*/
22  #ifndef __MAIN_H
23  #define __MAIN_H
24
25  #ifdef __cplusplus
26  extern "C" {
27  #endif
28
29  /* Includes -----*/
30  #include "stm32f4xx_hal.h"
31
32  /* Private includes -----*/
33  /* USER CODE BEGIN Includes */
34
35  /* USER CODE END Includes */
36
37  /* Exported types -----*/
38  /* USER CODE BEGIN ET */
39
40  /* USER CODE END ET */
41
42  /* Exported constants -----*/
43  /* USER CODE BEGIN EC */
44
45  /* USER CODE END EC */
46
47  /* Exported macro -----*/
48  /* USER CODE BEGIN EM */
49
50  /* USER CODE END EM */
51
52  void HAL_TIM_MspPostInit(TIM_HandleTypeDef *htim);
```

```

53
54 /* Exported functions prototypes -----*/
55 void Error_Handler(void);
56
57 /* USER CODE BEGIN EFP */
58
59 /* USER CODE END EFP */
60
61 /* Private defines -----*/
62 #define USART_TX_Pin GPIO_PIN_2
63 #define USART_TX_GPIO_Port GPIOA
64 #define USART_RX_Pin GPIO_PIN_3
65 #define USART_RX_GPIO_Port GPIOA
66 #define LD2_Pin GPIO_PIN_5
67 #define LD2_GPIO_Port GPIOA
68
69 /* USER CODE BEGIN Private defines */
70
71 /* USER CODE END Private defines */
72
73 #ifdef __cplusplus
74 }
75 #endif
76
77 #endif /* __MAIN_H */

```

Listado 4.1: Archivo de cabecera del programa principal, para incluir en main.c. Ambos se encargan de configurar los pines de la placa, timers, etc. (**main.h**)

Reproducimos aquí el código de **main.h**, porque nos va a permitir explicar algunas características importantes de los archivos fuente generados por STM32CubeMX. si nos fijamos, el archivo está dividido en secciones: includes, types, constants, macro, function protipes, etc. Además, en todas estas secciones aparece repetida un área, delimitada por los comentarios,

```
/* USER CODE BEGIN (abreviatura de la sección) */
```

```
/* USER CODE END (abreviatura de la sección)*/
```

Estas secciones están pensadas para que el usuario añada en ellas su propio código. La razón de definir las así, tiene que ver precisamente con STM32CubeMX. Supongamos que, tras generar los ficheros de código con STM32CubeMX y añadir nuestro código en ellos, nos damos cuenta de que no hemos configurado algo correctamente, o queremos añadir algo más, por ejemplo habilitar el uso de un pin nuevo. Para hacerlo bastaría con abrir de nuevo el archivo .ioc de nuestro proyecto en STM32CubeMX, corregir la configuración, o añadirle la nueva funcionalidad y volver a generar el proyecto. Al hacerlo, STM32CubeMX nos genera de nuevo los archivos del proyecto, conservando sin modificación el código que hayamos escrito en las secciones reservadas para el usuario. Pero si hemos escrito algo de código fuera de dichas secciones, STM32CubeMx lo eliminará de los nuevos archivos generados.

Este archivo de cabecera, incluye a su vez el archivo de cabecera **#include "stm32f4xx_hal.h"**. Es la entrada principal de acceso al hardware. Además define los prototipos de las funciones,

HAL_TIM_MspPostInit(TIM_HandleTypeDef *htim) y **Error_Handler(void)**. La primera permite relacionar los timers con los pines que los emplean y la segunda se encarga de gestionar errores.

El archivo además incluye unos **define** que asignan nombres específicos a los pines reservados para el puerto serie. En realidad nosotros no usaremos estos pines, ya que vamos a emplear el puerto USB de la placa para comunicarnos con el pc, emulando un puerto serie. Sin embargo, los pines asociados al puerto serie correspondientes quedan configurados en cualquier caso. Es interesante fijarse aquí como se definen. Cada pin tiene asociado un *puerto* (A, B, C ó D) y un número. Así por ejemplo, el **define** de la línea 62 define el pin 2 como pin de envío (**USART_TX_pin**) del puerto serie y el de la línea 23 establece que se trata del puerto A, por tanto, la configuración completa lo define como el pin **A2**.

Aparte de configurar los pines del puerto, también se configura el pin **5A** como el pin asociado al led 2 (L2, led verde) de la placa núcleo. Este pin no es parte del sistema de control de motor, pero lo encenderemos, cuando arranquemos la placa, como testigo de que el código se está ejecutando.

4.2.2. main.c

Este archivo contiene el bucle principal del programa que ejecutará el microprocesador así como las definiciones de todas las funciones, relacionadas con el Hardware: Lectura de encoders, generación de señales de PWM, comunicaciones con el pc, etc. Vamos a ver en detalle como está implementado. No vamos a reproducir aquí todo su código. Para entender esta descripción es conveniente tener delante el archivo del código.

El código incluye tres archivos de cabecera: `main.h`, del que ya hemos hablado, `math.h` que es un archivo de cabecera que da acceso a funciones matemáticas estándar y `motor.h`. Este último contiene las definiciones de las funciones que vamos a emplear para controlar el motor. Lo veremos detenidamente en más adelante.

A continuación definimos dos estructuras de datos. Ambas están pensadas para intercambiar información con el pc a través del puerto serie,

```

1 typedef struct {
2     uint32_t id; //ID para identificar el tipo de dato 1 para direccion, 2 para tiempo y
3     posicion)
4     uint32_t tiempo_ms; //tiempo o direccion dependiendo del ID
5     int32_t posicion; //posicion del encoder (solo se utiliza si el ID es 2)
6     float32_t velocidad; //velocidad del motor
7     float32_t velocidad_est; //velocidad estimada por el observador de espacio de estados
8 } DatosPck;
9
10 typedef struct{
11     uint32_t id; //ID para identificar el tipo de dato 1 para direccion, 2 para PWM)
12     int32_t valor; //valor del comando (puede ser el valor de PWM, la direccion, etc.
13     dependiendo del ID)
14 } ComandoPck;

```

Listado 4.2: Estructuras de datos para enviar información al pc y recibir comandos desde éste a través del puerto serie

La primera estructura `DatosPck`, Contiene información del estado del motor que la placa ha recopilado, medido o calculado. La primera variable es un identificador, que permite definir la naturaleza del mensaje que va a enviar la placa al pc. Solo son necesarios, en la configuración actual, dos tipos de mensajes. Si hacemos `id = 1` nuestro mensaje indica que ha habido un cambio en la dirección de movimiento del motor. Este cambio se ha activado directamente por hardware: pulsando el botón azul de la placa. Se informa al pc de la nueva dirección. En otro caso, `id = 2`, se está transmitiendo información de la posición (encoder) y la velocidad del motor, en el momento del envío. El nombre de las variables es autoexplicativo (ver listado 4.2). La última variable, `velocidad_est` corresponde a la velocidad calculada por el estimador de Luenberger cuando se está empleando control por realimentación de estados estimados.

La segunda estructura `ComandoPck`, permite recibir órdenes desde el pc a través del puerto serie. La primera variable `id` permite identificar el comando, y la segunda nos da su valor. Los posibles comandos se detallarán más adelante.

A continuación `stm32cubemx`, dentro del grupo de variables privadas, ha definido unas estructuras especiales, tipo `TIM_HandleTypeDef`, para guardar parámetros relacionados con los temporizadores (timers). Define tres;

- **htim2. Lectura del Encoder (TIM2):** Se configura un temporizador en modo *Encoder Mode*. El hardware del STM32 se encargará automáticamente de decodificar las señales en cuadratura (Canal A y Canal B) y actualizar un registro contador (CNT), liberando a la CPU de esta tarea.
- **htim3 Generación de PWM (TIM3):** Se configura un temporizador para generar una señal PWM (Modulación por Ancho de Pulso).
- **htim4. Bucle de Control (TIM4):** Un temporizador configurado para generar una interrupción periódica exacta cada 1.0ms ($T_s = 0,001$ s). Esta interrupción garantiza que nuestro bucle de control se ejecute de forma determinista.

También, define un segundo grupo de variables, que contiene las propiedades del puerto serie (USART2) y los dos canales de DMA, para envío y recepción de datos

- **huart2 Comunicaciones:** Se habilita el puerto serie asíncrono configurado a 115200 baudios
- **hdma_usart2_tx Canal DMA:** para envío de Datos
- **hdma_usart2_rx Canal DMA:** para recepción de Datos

Cada una de estas estructuras contiene los datos de las configuraciones que definimos en `stm32cubemx`.

Además, se han definido otras dos variables privadas. Se les ha añadido el atributo `volatile`, ya que son variables que pueden ser accedidas y modificadas desde interrupciones. Son dos,

- `flag_timer`. EL timer TIM4 activa esta variable cada 1.0 ms, entrando en la condición de control del motor
- `flag_boton_pulsado`. Se activa por una interrupción asociada a que se haya pulsado el botón azul de la placa

Por último antes de entrar a la función principal, aparecen las declaraciones de las funciones de inicio de los pines, los timers, DMA y puerto serie. Son generadas automáticamente por STM32CubeMx y su definición se puede encontrar al final de este mismo archivo `main`.

Arquitectura del Firmware (STM32F4)

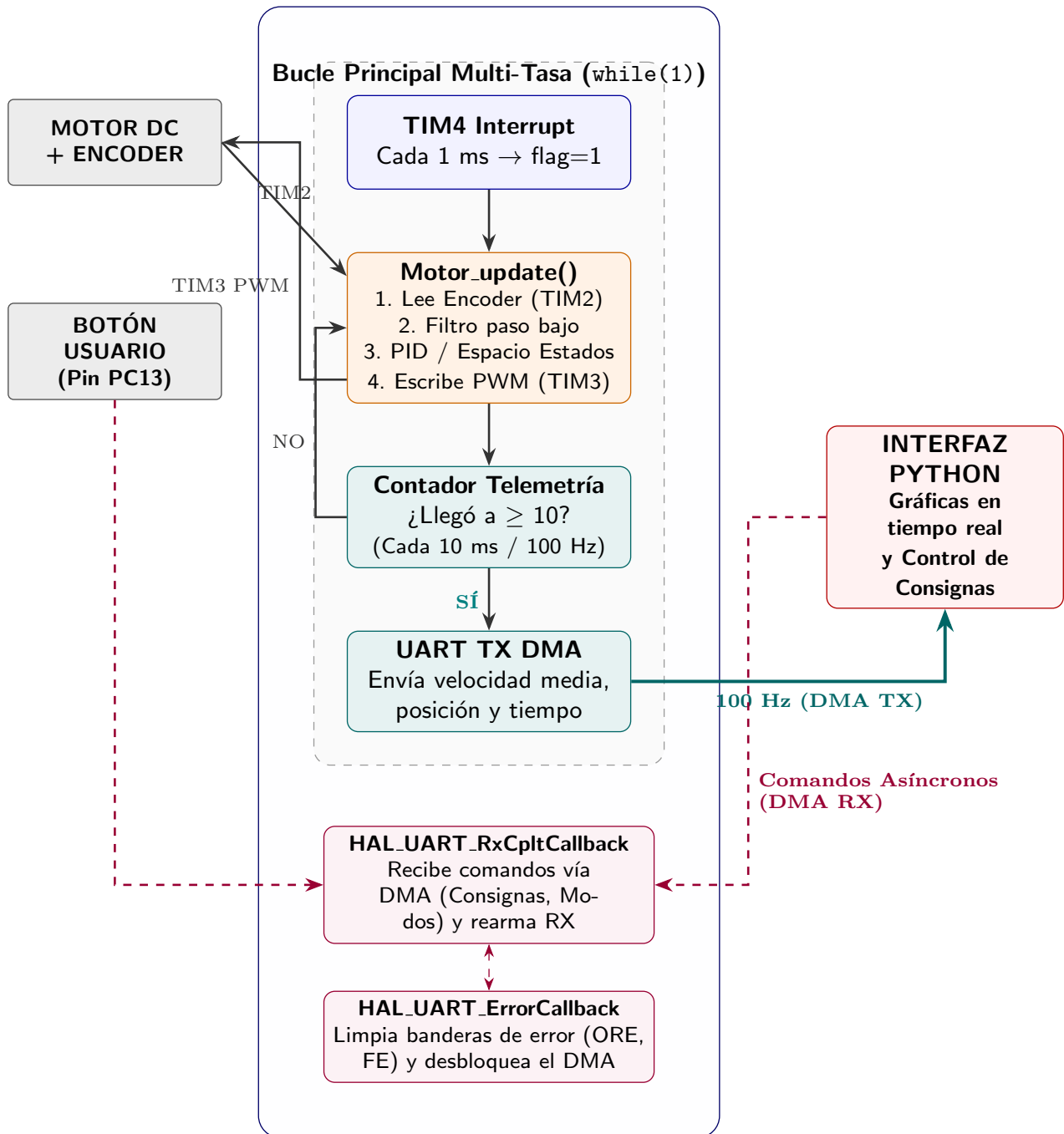


Figura 4.1: Arquitectura modular y síncrona del software de control del motor sobre la placa STM32.

main.c: Función main Entramos ahora a la descripción de la función main. En primer lugar se hacen llamadas a funciones que preparan la placa y la configuran de acuerdo con las características que se definieron empleando STM32CubeMx. Así se empieza reseteando los valores de configuración de todos los periféricos de la placa con la función HAL_Ini(). Se configura el reloj del sistema con la función SystemClock_Config(). Se inicializan los pines, cada uno con la función asignada usando MX_GPIO_Init(), Los canales de DMA: MX_DMA_Init(), Los timers: MX_TIMx_Init() y el puerto serie: MX_USART2_UART_Init().

A continuación se llama a la función MotorControl_ini(). Esta función inicializa los parámetros del sistema de control del motor se describirá con detalle en la sección 4.3.

A continuación se configuran los timers de acuerdo a la función asignada, empleando para en cada caso la función específica para ello:

1. HAL_TIM_Encoder_Start se aplica al timer dos que es el que se encargará de contar los pasos de encoder
2. HAL_TIM_PWM_Start Se aplica al timer tres que controlará la señal de PWM empleada para ajustar la tensión que se suministra al motor a través de la placa de expansión.
3. HAL_TIM_Base_Start_IT Se aplica al timer cuatro para que nos lleve el tiempo transcurrido

A continuación encendemos el led de la placa para que nos sirva de testigo de que el sistema está funcionando HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET).

Activamos la recepción de datos por el puerto serie en modo DMA. HAL_UART_Receive_DMA(&huart2, (uint8_t*)&rx_data, sizeof(rx_data)). Esta función queda a la escucha de los paquetes de datos que lleguen desde el ordenador a través del puerto serie. Implementa un sistema de Acceso Directo a Memoria (DMA). Es decir, la recepción se realiza almacenando directamente en memoria los datos en la estructura que creamos rx_data, sin interferir con la CPU que sigue ejecutando su código sin atender a esta recepción.

Cuando la recepción termina, se dispara una interrupción que llama automáticamente a la función HAL_UART_RxCpltCallback(&huart2), definida más abajo, en la sección de código USER_CODE 4 que contiene los callbacks del usuario y descrita en la sección 4.2.2.

Por último damos valores a los pines de configuración de la placa de expansión. Ponemos a cero el pin tres, para asegurar una tensión en el motor que lo haga girar en sentido 'positivo' y ponemos a uno los pines 7 y 10 para activar el puente en H.

Con esto acabaría la parte de configuración del código. Antes de entrar en el buche principal del programa, definimos e inicializamos a cero dos variables que vamos a emplear para fijar los tiempos en que enviaremos datos por el puerto serie al ordenador: contador_telemetría y para guardar los valores de la posición enviada por última vez: posicion_anterior_telemetria.

Entramos ahora en el bucle principal. Se trata de un bucle que repetirá su ejecución de modo indefinido desde que se suministra tensión a la placa y se ejecutan los pasos anteriores de inicialización hasta que se corte la tensión suministrada.

En este bloque podemos distinguir dos partes principales cada una asociada a una condición.

Botón pulsado La primera es una parte asíncrona que solo se ejecuta si se pulsa el botón azul de la placa. Cuando se pulsa el botón este genera una interrupción y ejecuta la función HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin). Esta función gestiona las interrupciones lanzadas por hardware desde cualquier pin. Para explicar mejor lo que hace, incluimos a continuación su código,

```

1 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
2 {
3     static uint32_t ultimo_tiempo_pulsacion = 0;
4     uint32_t tiempo_actual = HAL_GetTick();
5
6     if(GPIO_Pin == GPIO_PIN_13)
7     {
8         if (tiempo_actual - ultimo_tiempo_pulsacion > 200)
9         {
10             motor_pwm_output = -motor_pwm_output;
11             flag_boton_pulsado = 1; //Activar el flag para indicar que el boton ha sido
12             pulsado
13             ultimo_tiempo_pulsacion = tiempo_actual;
14         }
15     }
16 }

```

Listado 4.3: función HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)

La función lo primero que hace es crear la variable ultimo tiempo de pulsación y ponerla a cero. inmediatamente después toma el tiempo actual del reloj del sistema con la función HAL_GetTick. Comprueba que efectivamente el pin que ha provocado a interrupción es el pin 13 y a continuación comprueba el tiempo transcurrido desde la última pulsación. El objetivo de esta comprobación es evitar el efecto 'rebote'; a veces, al pulsar

y soltar el botón, este puede hacer contacto muchas veces. Si esto pasa, la función se llama de nuevo pero ahora el intervalo de tiempo transcurrido será menor de 1000 milisegundos.

Al definir la variable `ultimo.tiempo.pulsacion` con `static` sabemos que solo puede alterar su valor la asignación del valor de `tiempo.actual`, realizada dentro de la condición. Por tanto, aunque se llame a la función muchas veces por efecto del rebote esta no hará nada a menos que entre dos pulsaciones consecutivas hayan transcurrido 200 ms. Si se cumple la condición se cambia de signo a la variable `motor.pwm.output`, que, como se verá al describir el controlador del motor, es equivalente a cambiar de signo la tensión que le llega al motor y, por tanto cambiar de signo su sentido de giro. Además se activa un flag. Es decir, se pone a uno la variable `flag.boton.pulsado`.

Volvamos al bucle principal, solo si se ha pulsado el botón azul, la variable `flag.boton.pulsado` toma valor 1 y se ejecuta el código incluido dentro de la primera condición del bucle. Lo primero que se hace es volver a poner a cero el flag. A continuación se lee el valor de pin PB3, para determinar el sentido de giro actual del motor, y se crea un paquete de datos con esta información para enviarlo al ordenador a través del puerto serie. El paquete de datos es una estructura tipo `DatosPck`, el nombre de la estructura es `pktdir` y solo son relevantes sus dos primeros campos `pktdir.id = 1`, que indica que el mensaje que se va a enviar contiene la dirección de giro del motor y `pktdir.tiempo.ms = estado.dir`, que en este caso, no contiene un tiempo sino el estado de giro del motor que acabamos de cambiar al pulsar el motor.

Por último, se envía el mensaje empleando la función `HAL_UART_Transmit`. Esta función es capaz de enviar un mensaje byte a byte. Las variables que emplea son `&huar2`: dirección (software) del puerto serie 2. A continuación hace un casting de modo que crea puntero a bytes (`uint8_t*`) a la dirección donde se guarda nuestra estructura `pktdir`. El efecto es leer dicha estructura como si fuera un array de bytes. La siguiente variable `sizeof(pktdir)`, le dice a la función donde termina la estructura y, por tanto hasta donde debe leer para transmitir. La variable final es peligrosa: `HAL_MAX_DELAY` le está diciendo a la CPU que no haga nada hasta que termine la transmisión del mensaje. Así que, en caso de fallo podría bloquear el microcontrolador. Lo correcto sería emplear también un canal de acceso directo a memoria (DMA), como se hace con los mensajes de telemetría. Sin embargo, el uso del botón de la placa para cambiar la dirección de giro es algo poco frecuente y podemos correr el riesgo.

Temporizador Este temporizador es el que lleva el ritmo al que se controla el motor. Para ello, emplea el timer TIM4 que genera una interrupción cada 1ms, llama a su función de callback (ver listado 4.4). El callback se limita a asignar un valor uno a la variable `flag.timer`. Esto hace que a intervalos regulares de 1ms el valor que tiene la variable `flag.timer` sea un uno y, por tanto se cumpla la condición `if flag.timer`. El programa entra entonces en el código de esta segunda condición. pone a cero de nuevo la variable `if flag.timer` y llama inmediatamente a la función `Motor_update`. Esta función se describe en detalle en la sección 4.3 y es la que se encarga de actualizar la señal de pwm que se envía al motor de acuerdo con las consignas generadas por el sistema de control.

```
1 // 1. Callback del timer para actualizar el flag cada 1ms
2 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
3     if(htim->Instance == TIM4){
4         flag_timer = 1;
5     }
6 }
```

Listado 4.4: función callback empleada por el timer TM4

A continuación se incrementa el valor de `contador.telemetría` en una unidad. Cuando este contador llega a 10, es decir, cada 100ms, se entra en un nuevo condicional. Lo primero que se hace es poner de nuevo `contador.telemetría` a cero y, a continuación se llevan a cabo las siguientes tareas:

1. Se calculan cuantos pulsos de encoder han transcurrido durante los 10ms últimos, es decir, desde la vez anterior en que se entró en la condición `contador.telemetría`. Para ello, se lee el valor de la cuenta actual de encoder, `motor.posicion.actual` (ver descripción de `motor.c` y `motor.h`) e la sección 4.3, que ha sido refrescada al llamar a `Motor_update()` al entrar en la condición anterior del temporizador. Resta a ésta cantidad, la medida anterior `posicion.anterior.telemetría` y guarda el resultado en la variable `difpos.tel`.
2. Se crea una estructura `pck.envio` tipo `DatosPck`, para empaquetar la información de telemetría que se va a enviar por el puerto serie. Se declara como static, porque en C, una condición define un ámbito para las variables. Es decir si no se definiera como static, la estructura se destruiría al terminar al salir de la condición. Sin embargo, necesitamos que persista ya que el puerto serie va a acceder a su contenido en memoria empleando un canal de DMA. Si la variable no se conserva, el puerto serie podría enviar basura al corromperse la estructura.

Los valores asignados a `pck.envio` son: `id=2`, es decir, identificamos el paquete como un paquete de datos de telemetría (posición, velocidad, valores estimados). A continuación se define el campo `tiempo.ms`

tomado la medida del reloj del sistema `HAL.GetTick()`, el campo `posicion`, con el valor de la variable `motor.posicion.actual`, un valor de la velocidad promedio de los últimos 100ms `velocidad = dispos_tel*100`. Dado que el tiempo está medido en ms. mutiplicar la diferencia de posiciones por 100, nos daría el número de pasos (grados) avanzados por segundo.

Si estamos controlando el motor empleando variables de estado estimadas, enviamos también el valor de la velocidad calculada por el estimador, `velocidad_est = motor.x_hat[1]` (ver descripción en la sección 4.3). Si no estamos controlando mediante variables de estado estimadas, dejamos este último campo de la estructura `pck_envio` a cero.

por último, comprobamos si el puerto serie está libre y enviamos la estructura empleando un canal de DMA. Empleamos para ello el mismo método que empleamos para el envío del estado del botón de dirección; hacemos un casting de la estructura `pck_envio` a bytes, pero ahora usamos la función específica para emplear UART a través d DMA. El fragmento de código mostrado en el listado 4.5 muestra el uso de ésta función.

```
1 // Solo disparamos el DMA si no esta ocupada enviando el paquete anterior
2     if (huart2.gState == HAL_UART_STATE_READY) {
3         HAL_UART_Transmit_DMA(&huart2, (uint8_t*)&pck_envio, sizeof(pck_envio));
4     }
```

Listado 4.5: Protección de puerto serie libre y función de envío para USAR a través de DMA

Es interesante hacer notar que estamos trabajando a dos ritmos: el control se actualiza cada 1ms (1KHz), y la telemetría se envía al ordenador cada 10ms (100Hz).

Aquí termina el contenido del bucle y la función `main`.

USER CODE 4 En la sección de código contenida entre `USER CODE BEGIN 4` y `USER CODE END 4` Se contienen las funciones de callback definidas para gestionar las interrupciones definidas en el programa. Ya hemos descrito la asociada al periodo de envío de telemetría al ordenador y la generada cuando se pulsa el botón de usuario de la placa.

Nos quedan por definir dos funciones de callback más, asociadas a la recepción de datos a través del puerto serie.

Como se indicó más arriba, esta recepción se realiza empleando un canal de DMA. Cuando la recepción se ha completado, se genera una interrupción, para gestionar los datos recibidos. Esta gestión la lleva a cabo la función `HAL_UART_RxCpltCallback` Es una función genérica común a todos los puerto series, por ello, lo primero que hace es comprobar que la llamada proviene del USAR2, que es el que nosotros estamos empleando para recibir datos desde el ordenador. Una vez comprobado el puerto empieza a analizar el contenido de la estructura `rx_data`. En funcion del valor campo `rx_data.id` se implemente un arbol de condiciones que tiene la siguiente estructura:

1. `rx_data.id == 1` Corresponde a un comando enviado desde el ordenador para poner el motor en modo manual y definir/cambiar su dirección de giro. Pr tanto se ajuste el modo del motor `motor.mode = Control_pwm_openloop` y se examina a continuación el campo `rx_data.valor` Si toma valor cero, lo que se quiere es que el motor gire en sentido horario con lo que se ajusta el campo `motor.pwm_output` al valor absoluto que tuviera la señal de PWM multiplicado por menos uno. Si no toma valor cero (tomaría valor uno) se ajusta el campo directamente al valor de la seña de PWM.
2. `rx_data.id == 2` Esta opción tambien corresponde a control manual. En esta caso, lo que se cambia es el valor de la señal de PWM pero no el sentido de giro del motor. Se pone el modo del motor en manual igual que en el caso anterior y según el signo que tenga el valor actual de `motor.pwm_output` se ajusta su valor a `motor.pwm_output = -(float32_t)rx_data.valor` o a `motor.pwm_output = (float32_t)rx_data.valor`
3. `rx_data.id == 3` La opción tres, corresponde a control de posición mediante el uso de un PID. Se cambia el modo del motor `motor.mode = Control_posicion` y se carga el valor de consigna deseado `motor.setpoint_position = rx_data.valor`
4. `rx_data.id == 4` Esta opción corresponde con el control en velocidad mediante el uso de un PID. El procedimiento es análogo al anterior: `motor.mode = Control_speed` y `motor.setpoint_speed = (float32_t)rx_data.valor`
5. `rx_data.id == 5` Este modo corresponde a hacer un reset al motor. el resultado es que el motir se para y se resetean los valores de posición, velocidad, señal de PWM, consigna de posición, consigna de velocidad y estimadores de estados. también se resetean los valotres de los estados de los controladores PID.
6. `rx_data.id == 6` Este caso permite el control de posición del motor empleando para ello un observador de estados y control por realimentación de estados. En este caso el modo del motor se actualiza a `motor.mode = Control_state_space` y se fija la consigna de posición del motor `motor.setpoint_position = rx_data.valor`

7. `rx_data.id == 7` El último de los casos ajusta el modo a control de velocidad empleando realimentación de estados y ajusta la consigna de velocidad al valor indicado `motor.mode = Control_Speed_state_space` y se fija la consigna de posición del motor `motor.setpoint_speed = (float32_t)rx_data.valor`

Una vez seleccionada la opción y realizados los ajustes se vuelve a llamar a la función `HAL_UART_Receive_DMA` para que vuelva a armar la recepción a través de DMA.

El resto del código que contiene el archivo, son funciones de inicialización de periféricos y configuración de la placa y han sido generadas directamente por STM32CubeMX.

4.3. motor.h y motor.c

Los ficheros `motor.h` y `motor.c` Contienen las funciones necesarias para controlar el motor. Vamos a empezar por describir el archivo de cabecera.

4.3.1. motor.h

En primer lugar, incluimos el archivo de cabecera `arm_math.h` que nos da acceso a funciones matemáticas y algunos tipos de controladores. Al configurar nuestra placa con STM32CubeMX, pedimos que se incluyera la librería ARM CMSIS-DSP (Cortex Microcontroller Software Interface Standard - Digital Signal Processing). Esta librería contiene un conjunto de funciones optimizadas desarrolladas por ARM para explotar al máximo el hardware de los microcontroladores Cortex-M (como el de la placa STM32F411E). Está diseñada específicamente para aplicaciones de procesamiento digital de señales (DSP) y control en tiempo real.

A continuación se crea una enumeración, una estructura y los prototipos de las funciones de control que se definirán en `motor.c`. Vamos a describir solo los tipos ya que les funciones las describiremos en detalle al hablar de este último archivo.

El listado 4.6 muestra los tipos definidos.

```
1 typedef enum{
2     Control_pwm_openloop, //manual
3     Control_position, //PID de posicion
4     Control_speed, //PID de velocidad
5     Control_state_space, //Controlador de espacio de estados posicion
6     Control_speed_state_space //Controlador de espacio de estados velocidad
7 } MotorControlMode;
8
9 typedef struct{
10     //Modo de control actual
11     MotorControlMode mode;
12     //PID
13     arm_pid_instance_f32 pid_position;
14     arm_pid_instance_f32 pid_speed;
15
16     //Controlador de espacio de estados
17     float32_t x_hat[2]; //Estado estimado [posicion; velocidad]
18     float32_t x_integral_pos; //Integral para control de posicion
19     float32_t x_integral_speed; //Integral para control de velocidad
20     //estado del motor
21     int32_t posicion_actual;
22     int32_t posicion_anterior;
23     float32_t velocidad_actual;
24     //setpoints
25     int32_t setpoint_position;
26     float32_t setpoint_speed;
27     //Salida PWM
28     float32_t pwm_output;
29 } MotorControl;
```

Listado 4.6: Tipos definidos en `motor.h`

La enumeración permite definir cinco 'modos distintos de control del motor. el primero de ellos `Control_pwm_openloop` corresponde a control manual. es el usuario quien, a través el puerto serie, fijar el 'Duty Cycle' de la señal de pwm que se envía a la placa de control y, por tanto, el voltaje que se suministra la motor. El segundo modo `Control_posicion` Corresponde a un modo de control automático de la posición del motor, mediante el empleo de un control proporcional-derivativo-integral (PID). De modo análogola, el modo `Control_speed` permite controlar la velocidad a la que gira el eje cel motor, emplenado de nuevo u PID. LOs dos últimos modos `Control_state_space` y `Control_speed_state_space` permiten controlar la posición y al velocidad del motor empleando realimentación por variables de estado estimadas. El tipo creado recibe el nombre `MotorControlMode`

En cuanto a la estructura se emplea para agrupar las variables necesarias así como el estado del controlador. Sus campos son:

1. Un modo de control (`mode` definido mediante un enumerador del tipo `MotorcontrolMode` que acabamos de definir.
2. Dos estructuras tipo `arm_pid_instance_f32`. Estas estructuras sirven para guardar los valores asignados a las constantes proporcional, derivativa e integral de un PID. La primera la llamamos `pid.position`. Nos servirá para guardar los valores del PID cuando queramos realizar control en posición. La segunda la llamamos `pid.speed` y tendrá idéntico uso pero ahora cuando queramos controlar la velocidad del motor.
3. Variables necesarias para definir los estados del estimador `x_hat[2]` un array de dos elementos para contener la posición estimada y la velocidad estimada.
4. Dos variables de estado extra para implementar control integral: `x.integral_pos` para el error en la posición y `x.integral_speed` para el error en la velocidad.
5. Tres variables más para almacenar el estado del motor: `posicion_actual`, `posicion_anterior` y `velocidad_actual`.
6. Dos variables más para almacenar los valores de consigna asignados a la posición `setpoint_position` y `setpoint_speed`.
7. Por último, una variable para guardar la señal de pwm que se enviará a la controladora del motor `pwm_output`.

Definimos el tipo resultante como `MotorControl`. Además, declaramos una variable global `motor` del tipo que acabamos de definir. Al definirla como `extern` la hacemos accesible desde cualquier archivo que incluya a `motor.h`.

4.3.2. motor.c

Este es el archivo que contiene todo el sistema de control de nuestro motor. El fichero empieza con la inclusión de los ficheros de cabecera `motor.h` y `math.h`. Este último, en realidad no sería necesario ya que podrían emplearse las funciones de `arm_math.h` incluidas en `motor.h`. Pero lo dejaremos de momento para una futura revisión del código.

A continuación definimos la estructura `motor` que declaramos en `motor.h`, y declaramos como `extern` las dos estructuras `htim2` y `htim3` que definimos en `main.c`, para manejar los timers que controlan la lectura de encoders y la generación por hardware de las señales de PWM necesarias para dar tensión al motor.

El siguiente paso, es definir `TS_DSP 0.001f` y `VMAX_DSP 12.0f`, empleamos para ello la directiva `#define` ya que tanto la primera (tiempo de muestreo del sistema de control) como la segunda (Voltaje máximo del sistema) son valores fijos que nunca se alteran.

Las siguientes definiciones son fundamentales para el control en variables de estado, ya que definen los valores de las matrices de estado del sistema discretizado. El listado 4.7 las muestra con todos sus valores puestos a cero. Evidentemente, para que el sistema de control funcione estos valores deberán ajustarse de acuerdo con los valores calculados a partir del modelo obtenido para el motor (ver guión de la práctica.).

El sistema que modela el motor sigue una representación estándar en variables de estado,

$$\dot{x} = Ax + Bu \quad (4.1)$$

$$y = Cx \quad (4.2)$$

Que corresponden con las matrices `Ad_f32`, `Bd_f32`, `Cd_f32` del código.

```

1 // Matrices del modelo discretizado (A, B, C)
2 float32_t Ad_f32[4] = {0.0f, 0.0f, 0.0, 0.0f}; // Matriz A (2x2)
3 float32_t Bd_f32[2] = {0.0f, 0.0f}; // Matriz B (2x1)
4 float32_t Cd_f32[2] = {0.0f, 0.0f}; // Matriz C (1x2)
5
6 // Ganancias del controlador y observador de espacio de estados
7 float32_t Lp_f32[2] = {0.0f, 0.0f}; // observador de estado (L)
8 float32_t Kx_pos_f32[2] = {0.0f, 0.0f}; // Ganancia para control de posicion
9 float32_t Kx_vel_f32[2] = {0.0f, 0.0f}; // Ganancia para control de velocidad
10
11 float32_t Ki_pos_f32 = 0.0f; // Ganancia integral para control de posicion
12 float32_t Ki_vel_f32 = 0.0f; // Ganancia integral para control de velocidad
13
14 // Objetos (instancias) matrices de CMSIS-DSP
15 arm_matrix_instance_f32 Ad_mat, Bd_mat, Cd_mat, Lp_mat, Kx_pos_mat, Kx_vel_mat;
16 arm_matrix_instance_f32 x_hat_mat, y_hat_mat;
17 arm_matrix_instance_f32 temp1_mat, temp2_mat, temp3_mat;
18
19 float32_t y_hat_f32[1] = {0.0f};

```

```
20 float32_t temp1_f32[2], temp2_f32[2], temp3_f32[2];
```

Listado 4.7: Matrices del sistema discretizado

A continuación se define la ganancia de la realimentación de estado K , de la que se definen dos versiones. La primera, Kx_pos , para controlar el motor en posición y la segunda, Kx_vel para controlar el motor en velocidad. Se define también la ganancia correspondiente al estimador L . Además se definen dos ganancias Ki_pos y Ki_vel para añadir control integral al control en posición y al control en velocidad.

Para completar las variables necesarias, se define también una variable de salida para el estimador y_hat , que se inicializa a cero y tres variables auxiliares $temp1_f32$, $temp2_f32$, $temp3_f32$. Además, se definen 'objetos' matrices empleando los tipos propios de la librería CMSIS-DSP, de modo que hay una matriz CMSIS-DSP por cada una de las matrices definidas como arrays normales. Los nombres coinciden salvo por la terminación que ha pasado de ser $f32$ (float32) a mat .

Nota importante: Todos los elementos de las matrices así como las ganancias de los controles integrales tienen asignado valor cero. Es preciso darles valores adecuados para que los controladores funcionen, compilar el proyecto y descargar el resultado en la placa.

A continuación, empieza la definición de las funciones de control del motor,

MotorControl_ini() Esta primera es una función de inicialización que se llama al principio de la función **main()** del archivo **main.c**. Inicializa los dos modos posibles de control, dejándolos listos para su uso. Para ello emplea la estructura **motor** declarada al principio de este código. Las opciones de control disponibles son:

1. Control de posición mediante un PID. Permite llevar el eje del motor a una posición angular deseada empleando para ello controles Proporcional/Integral/Derivativo (PID). Para ello, se ajustan los valores de las ganancias proporcional **motor.pid_position.Kp**, integral **motor.pid_position.Ki** y derivativa **motor.pid_position.Kd**. Por defecto estos valores tienen valor cero, es decir NO están ajustados y el control no funciona. Es preciso darles valores adecuados, compilar el proyecto y descargar el software en la placa, para que el controlador funcione. Una vez definidas las constantes, se llama a la función **arm_pid_ini_f32**, de la librería CMSIS pasando como variables de entrada la dirección de **motor.pid_position** y un uno. Esto implementa un controlador PID y además resetea su estado.
2. Control de velocidad mediante un PID. análogo al anterior. El objetivo es ahora estabilizar la velocidad a la que gira el motor a un valor deseado. Se definen ahora los campos correspondientes, proporcional **motor.pid_speed.Kp**, integral **motor.pid_speed.Ki** y derivativa **motor.pid_speed.Kd**, para las ganancias del controlador para la velocidad. Al igual que en el caso de controlador en posición, los valores por defecto están ajustados a cero y es preciso introducir los valores que se quieran usar para controlar el motor. De nuevo, llamamos a **arm_pid_ini_f32** pasándole ahora la dirección de **motor.pid_speed**, para crear un controlador PID para la velocidad.
3. Control de posición usando variables de estado y un estimador. Las matrices necesarias ya se definieron como variables globales antes. Sin embargo, es preciso llamar a la función **arm_mat_init_f32**, para vincule las matrices (arrays) con los objetos matrices de la librería CMSIS-DSP, que hemos creado, para poder realizar cálculos con ellas de una forma eficiente.
4. Control de velocidad usando variables de estado y un estimador. Se vinculan las matrices creadas igual que en el caso anterior.

Además, se inicializa también el estado del motor, poniendo todas sus variables de estado a cero. De modo análogo se inicializan los estados del estimador y del error integral.

Motor_update() Esta segunda función es la que lleva todo el peso del control real del motor. Se llama desde el bucle principal de **main.c** cada 1ms, usando el timer **TIM4**.

Lo primero que hace es leer, empleando el contador del timer **TIM2** que controla los encoders, cual es la posición actual del motor y la guarda en **motor.posicion_actual**. Calcula cual es la diferencia entre éste valor y la posición anterior guardada en **motor.posicion_anterior** y, multiplicando esta diferencia por 1000, calcula cual es la velocidad del motor en grados/s. A continuación se emplea un filtro de media móvil para suavizar el valor de la velocidad,

$$v_m = 0,8v_m + 0,2v_l \quad (4.3)$$

Donde v_m representa la velocidad media y v_l la velocidad que se acaba de calcular. Los valores 0,8 y 0,2 son tentativos y pueden ajustarse para obtener un balance adecuado entre ruido y retardo.

El siguiente paso, es crear una variable **salida** que será la que contendrá el valor de la señal de PWM con la que modificaremos/ajustaremos el voltaje suministrado al motor para controlarlo.

La lógica empleada en el control variará dependiendo del modo `motor.mode` en que se encuentre el motor. Para seleccionarlo se emplea un juego de condicionales de acuerdo con el `mode`,

1. `motor.mode == Control_position`. Este modo corresponde al control de posición del motor empleando control PID. Se define una variable `error_pos` Calculando la diferencia entre la posición actual del motor y el valor de consigna (deseado), para la posición. El control, evita la banda muerta del motor, dejando de actuar si el error en posición es menor o igual a un grado. La salida se ajusta directamente, empleando la función `arm_pid_f32` de la librería CMSIS-DSP que implementa un PID. Esta función admite dos variables, la primera es un puntero a una estructura `arm_pid_instance_f32`. En nuestro caso, pasamos la dirección de `motor.pid_position`, que contiene los parámetros de nuestro controlador PID para posición. Además, le pasamos el como segunda variable el error en posición que acabamos de calcular¹
 Además se ha implementado un saturador por software de modo que si el valor de la salida es superior a 999 o inferior a -999, la salida se ajusta a dichos valores límite. El valor de la salida calculado, se guarda en la variable de estado del PDI `motor.pid_position.state[2]`. Por último, para evitar cualquier oscilación debida a ruido si el error de posición leído es estrictamente cero, la salida se pone a cero.
2. `motor.mode == Control_speed` El funcionamiento es idéntico al del caso anterior. Las dos diferencias principales son que el error en velocidad se calcula comparando la consigna de velocidad con el valor de la velocidad filtrado mediante el filtro de media móvil y que el primer parámetro que se pasa a la función `arm_pid_f32` es la dirección de `motor.pid_speed`, que contiene los parámetros del controlado PID para la velocidad.
3. `motor.mode == Control_state_space || motor.mode == Control_speed_state_space`² Esta opción activa los sistemas de control basados en variable de estado; tanto el de posición como el de velocidad. Lo primero que se hace es obtener el valor de las salidas estimadas: $\hat{y} = C_d \hat{x}$. Además se compara la posición actual medida que está disponible en `motor.posicion_actual` con la posición estimada para obtener el error del observador. Y se crea una variable `u_ideal_volts` para la entrada ideal y un par de matrices `u_Kx_mat`, `u_Kx_f32` para guardar el valor de la entrada que debemos aplicar al motor. Se examina a continuación el modo exacto en que nos encontramos,
 - a) `motor.mode == Control_state_space`. Estamos en un modo de control de posición.
 Comparamos el valor consigna con la posición estimada para obtener el error en posición. incrementamos el valor del error integral en posicion añadiendo el error de posición actual multiplicado por el tiempo de muestreo ($TS_{DSP} = 1ms$) `motor.x.integral_pos += error_seguimiento * TS_DSP`.
 Calculamos cuanto sería la entrada, multiplicado las ganancias de realimentacion por el vector de estados estimados $u = K \hat{x}$. A este resultado cambiado de signo le restamos la contribución debida al error integral y obtenemos el valor de la entrada que guardamos en la variable `u_ideal_volts`.
 Es interesante señalar, que el estimador que estamos empleado, que se ha obtenido identificando el motor, trabaja en voltios. Es decir las constantes están calculadas de modo que la entrada al sistema calculada `u_ideal_volts` se obtiene en voltios.
 - b) `else` Si no estamos en el modo de control de posición por realimentación de estados, solo podemos estar en el de velocidad. El cálculo es idéntico con la unica diferencia de que aquí empleamos la velocidad estimada en lugar de emplear la posición.

Una vez calculado el valor del voltaje de entrada al motor, saturamos su valor a los límites físicos del voltaje suministrado por la fuente $[+12V, -12V]$. Para ello, creamos una nueva variable `u_real_volts` que igualamos a `u_ideal_volts` si esta entre dichos límites o a la que asignamos valor 12 ó -12 si está fuera de ellos.

Por último aplicamos una corrección de antiwindup a la contribución del término integral. Hay muchas formas de implementarlo. Aquí, hemos obtenido por un método que se conoce como back calculation. La idea es hacer una corrección en la integral del error, de modo que se ajuste su valor al que habría tenido para dar el valor del voltaje máximo o mínimo posible.

Si nos fijamos en el cálculo de la entrada al sistema en función del error,

$$u_i = -K\hat{x} - K_i \int_0^T (\hat{y} - y_r) dt$$

¹No incluimos aquí como está implementado el controlador PID. Los interesados, puede encontrar toda la información en la documentación de la librería CMSIS-DSP

²El sistema que estamos describiendo, basado en condicionales, para seleccionar el modo de control aplicado al motor es muy pobre. Será mejorado en futuras versiones del código.

donde $\hat{x}$ representa los estados estimados y $\hat{y}$, y_r representa la salida (posición o velocidad) estimada y su valor de referencia. Sin embargo, sabemos que el voltaje que nos puede dar la fuente está limitado, de modo que si el valor obtenido de u_i supera los límites, ajustamos el valor a dichos límites u_{lim} . El método back calculation, consiste precisamente en descontar a la contribución del error integral la parte correspondiente a la diferencia entre la salida real aplicada u_{lim} y la salida inicialmente calculada u_i

$$u_{lim} = -K\hat{x} - K_i \int_0^T (\hat{y} - y_r)dt - u_i + u_{lim} = -K\hat{x} - K_i \left[\int_0^T (\hat{y} - y_r)dt + \frac{u_i - u_{lim}}{K_i} \right]$$

Para ello, en primer lugar comprobamos si las entradas `u_real_volts` y `u_ideal_volts` no coinciden. Si es así, quiere decir que el valor ideal está fuera del rango de voltaje aplicable al motor. Comprobamos a continuación si estamos controlando posición o velocidad, y modificamos la componente de error integral del controlador correspondiente, tal y como se muestra en el fragmento de código del listado 4.8 (líneas 1-8)

A continuación actualizamos la salida del observador, para poder emplearlo en el siguiente ciclo de control: $\hat{x}(n+1) = A_d\hat{x}(n) + B_d u(n) + L(y - \hat{y})$.

Esta operación hay que hacerla por partes, tal y como se muestra en el listado de código 4.8 (líneas 9-15). En primer lugar calculamos el producto de A_d por $\hat{x}$ y guardamos el resultado en la variable temporal `temp1_mat`. Hacemos lo mismo con B_d y la entrada real que vamos aplicar al motor y guardamos el resultado en `temp2_mat` y con L y el error de estimación y guardamos el resultado en `temp3_mat`. A continuación sumamos los tres resultados, la suma también debemos hacerla en dos pasos, ya que estamos empleando todo el tiempo las funciones de la librería CMSIS-DSP.

El último paso es generar la variable salida calculando el valor de la señal de PWM correspondiente al valor en voltios de la variable `u_real_volts`. Para el caso en que se esté realizando control de posición calculamos si la posición del motor está en el límite de un grado respecto a la posición deseada en cuyo caso, paramos directamente el motor.

4. **else.** Si no estamos en ninguno de los modos anteriores, entonces es que estamos en modo manual, en cuyo caso, `salida = motor.pwm_output`, damos a la salida el valor de la señal de PWM directamente contenido en el campo `pwm_out` de la estructura `motor`.

Tan solo nos queda enviar a los pines de la placa el valor de la señal de PWM y el sentido de giro del motor.

Si la salida es positiva, ponemos a uno el pin 3: `HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, GPIO_PIN_SET)` y Enviamos el valor de la variable `salida` al comparador del TIMER 3 que es el que está controlando el tiempo que el pin A6 está en el valor alto. Si la salida es negativa, lo que hacemos es poner a cero el pin 3 y enviar al comparador el valor de salida cambiado de signo.

```

1 // D) Anti-Windup (Back-Calculation Corregido)
2     if (u_real_volts != u_ideal_volts) {
3         if (motor.mode == Control_state_space) {
4             motor.x_integral_pos += (u_ideal_volts - u_real_volts) / Ki_pos_f32;
5         } else {
6             motor.x_integral_speed += (u_ideal_volts - u_real_volts) / Ki_vel_f32;
7         }
8     }
9 // E) Actualizar Observador: x_hat = Ad*x_hat + Bd*u_real + Lp*error_obs
10 arm_mat_mult_f32(&Ad_mat, &x_hat_mat, &temp1_mat);           // temp1 = Ad * x_hat
11 arm_mat_scale_f32(&Bd_mat, u_real_volts, &temp2_mat);        // temp2 = Bd * u_real
12 arm_mat_scale_f32(&Lp_mat, error_obs, &temp3_mat);           // temp3 = Lp * error_obs
13
14 arm_mat_add_f32(&temp1_mat, &temp2_mat, &x_hat_mat);          // x_hat = temp1 + temp2
15 arm_mat_add_f32(&x_hat_mat, &temp3_mat, &x_hat_mat);          // x_hat = x_hat + temp3

```

Listado 4.8: Corrección antiwindup

Capítulo 5

Programación en Python de un monitorizador.

Una vez terminada la descripción del código empleado por el microcontrolador para controlar el motor, nos queda describir el programa para manejarlo desde el ordenador.

En este caso, tenemos un solo programa `Motor.py`; un script de Python standard. Cuando se ejecuta, el script abre una ventana que muestra un panel de control (*Dashboard*), que permite al usuario interactuar con el motor y monitorizar la posición y la velocidad del motor. La figura 5.1 muestra una imagen del panel de control. Su uso se detalla en el guión de la práctica y no se repetirá aquí.

La ventana y los *gadgets* se han desarrollado usando Qt, que es un *framework* (conjunto de herramientas y bibliotecas) para desarrollar aplicaciones gráficas (interfaces de usuario, ventanas, botones, etc.) en C++, pero también está disponible para otros lenguajes como Python, Java, JavaScript, etc. PyQt es la versión de Qt para Python, es decir, un binding (enlace) que permite usar todas las funcionalidades de Qt directamente desde código Python. Si nos fijamos en la cabecera del módulo `Motor.py`, podemos ver los módulos importados: `serial`, para manejar el puerto serie, `struct` para la gestión de los paquetes de datos, `csv` para general ficheros `csv` con los datos recibidos desde el microcontrolador, `sys` para gestionar comoandos del sistema operativo del ordenador, `time` para medir tiempos, `collection` para generar estructuras de datos distintas de las estándar de python, `datetime`, para marcar fechas, `pyqtgraph`, `QtCore`, `QtWidgets` para genera objetos gráficos con Qt y `numpy` para realizar operaciones matemáticas.

Después de esto se definen dos variables globales, una de ellas es el nombre del puerto serie del ordenador donde se recibirán los datos procedentes del microcontrolador. Hay que recordar que en realidad, estos datos se están recibiendo a través de un puerto USB que está emulando un puerto serie. El nombre asignado `PUERTO = '/dev/ttyACM0'` suele ser el típico asignado en Linux por defecto, pero no es mala idea comprobar que efectivamente ese es el nombre con el que el sistema operativo ha montado el puerto¹.

La segunda variable es la velocidad de transmisión del puerto que se fijado 115200 baudios que es la misma velocidad a la que se ha configurado el puerto serie en el microcontrolador.

El siguiente paso, es crear una clase, llamada `MotorDashboard` que contiene dentro todos los elementos y funciones necesarios para levantar la ventana del panel de control.

Por último, si se ejecuta directamente `Motor.py`, se genera objeto tipo `MotorDashboard`, llamando al constructor de la clase, y se abre en el ordenador la ventana del panel de control `dashboar.show()`, lista para que el usuario la emplee. Cuando el usuario cierra la ventana el ordenador termina la ejecución del código y libera la memoria `sys.exit(app.exec_())`.

Veamos ahora despacio la clase `MotorDashboard`.

5.1. MotorDashboard

La clase `MotorDashboard` contiene todas las funciones necesarias para crear el panel de control con el que el usuario puede interactuar con el motor a través de la placa de control, monitorizar las salidas (posición angular del rotor y velocidad) y almacenar datos en archivos tipo `csv`. En la definición se pasa como parámetro la clase `QtWidgets.QMainWindow`, de modo que `MotorDashboar` heredará de ésta todos sus métodos y atributos. Esta clase, esta asociada a Qt y permite crear la ventana principal de la aplicación y emplear todas las funciones y atributos asociados a las ventanas de Qt.

¹para comprobarlo basta ir al directorio `/dev` y enchufar y desenchufar el cable USB, listando los archivos disponibles con `ls` deberá aparecer y desaparece `ttyACM0` o el nombre del archivo donde el sistema lo ha montado

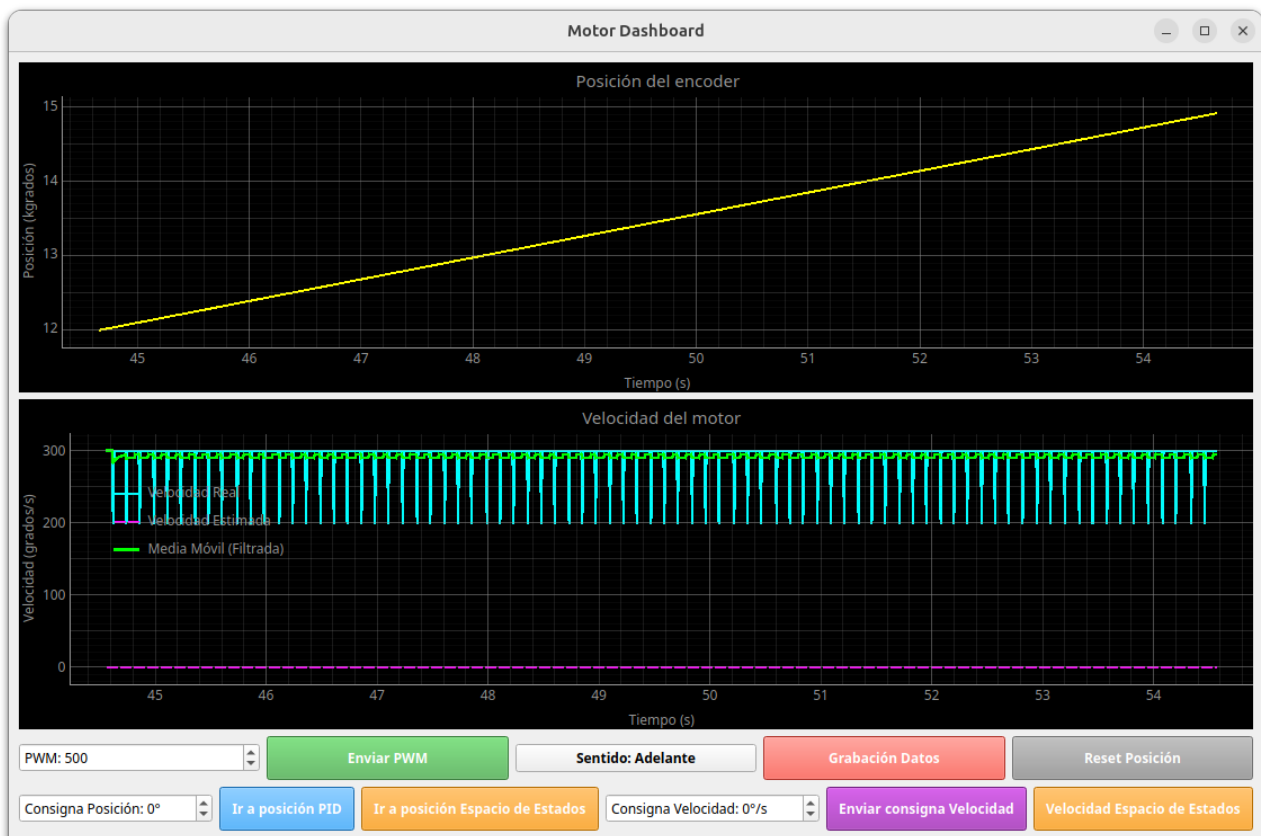


Figura 5.1: Vista del panel de control desarrollado para interactuar con el motor

El constructor. Lo primero que hace el constructor de la clase, es llamar al constructor de la clase superior QMainWindow, de este modo se crea la ventana principal del panel de control, Además empleando los métodos setTitle y resize, Da título a la ventana creada y le asigna un tamaño por defecto.

El siguiente paso es configurar el puerto serie. Para ello trata de crear un objeto Serial con el nombre del puerto y la velocidad (baudios) definidos previamente. Si no es capaz de establecer la comunicación, se cierra la aplicación y se nos muestra un mensaje de error en el terminal.